PDF Tools

This document describes ragent's **PDF document tools** – the `pdf_read` and `pdf_write` families. It covers formats, JSON input schemas, worked examples, permissions, checkpointing, and configuration options.

Both tools parse and write **in-process with pure-Rust crates**. There is no `soffice`/LibreOffice binary, no Microsoft Word, and no external converter to install. A workspace-wide grep for `soffice` returns zero matches.

> **History:** earlier ragent releases also shipped MS Office (`office_*`) and LibreOffice/OpenDocument (`libre_*`) document tools. Those families have been removed; only the PDF family remains. See Section 3 for which document formats ragent still understands.

Contents

1. Tool inventory
2. Enabling the tools
3. Format matrix
4. Tool reference
5. Output formats and truncation
6. Permissions and checkpointing
7. Implementation notes
8. Worked examples
9. Commands and configuration reference
10. Testing and sample files
11. Related documentation

---

1. Tool inventory

Two tools are registered by `crates/ragent-tools-extended/src/lib.rs`:

Tool name	Family	Purpose	Permission
`pdf_read`	PDF	Extract text or metadata from PDF files	`file:read`
`pdf_write`	PDF	Create new PDF documents	`file:write`

Both are implemented in `crates/ragent-tools-extended/src/` (modules `pdf_read.rs`, `pdf_write.rs`, plus the shared helper module `pdf_common.rs`).

---

2. Enabling the tools

The PDF family needs no switch: both tools register at startup and every registered tool is advertised to the model. `/tools list` shows them.

3. Format matrix

Family	Formats supported	Legacy formats
`pdf_read`	`.pdf`	–

`pdf_read` rejects files that are not PDFs, and `pdf_write` always emits a `.pdf`. Plain-text and markdown extraction (used by the research `--from-file` pre-step) is handled by `crates/ragent-tools-extended/src/document_ex` which reads `.pdf` through `pdf_read` and `.md`/`.markdown`/`.txt` verbatim.

Legacy binary and OOXML/ODF document formats (`.doc`, `.xls`, `.ppt`, `.docx`, `.xlsx`, `.pptx`, `.odt`, `.ods`, `.odp`) are **not** supported for extraction and return an actionable error.

4. Tool reference

4.1 `pdf_read`

Extracts text from PDF files in-process.

Parameter	Type	Required	Notes
`path`	string	**yes**	Path to the PDF
`start_page`	integer	no	1-based, inclusive
`end_page`	integer	no	1-based, inclusive
`format`	enum	no	`text` (default), `metadata`, or `json`

Page extraction works by extracting the whole document and filtering to the requested range, using `lopdf` page content streams with a whole-document `pdf-extract` fallback when per-page extraction yields empty pages. The `metadata` format returns path/page-count/title/author/subject/creator/producer fields; the `json` format returns structured per-page text plus metadata. The `text` format's reported line count reflects the text *after* truncation (Section 5).

Encrypted PDFs are not supported: the vendored `pdf-extract` crate only decrypts through password-taking variants that `pdf_read` does not use, so an encrypted document errors out.

4.2 `pdf_write`

Creates a new PDF from a structured payload. Requires `path` + `content`; the `content` object has an optional `title` and an `elements` array whose items are `{type: "paragraph" | "heading" | "table" | "image", text, level, headers, rows, image_path, width_mm, caption}` (`level` 1-3 for headings; `headers`/`rows` for tables; `image_path` for images). Layout is fixed A4 (210x297 mm) with 25 mm margins and built-in font sizes (11 pt body, 22 pt H1, 17 pt H2, 13 pt H3). Parent directories are created automatically, and the resolved output path is confined to the workspace root.

5. Output formats and truncation

- Successful reads truncate at `MAX_OUTPUT_BYTES` = **100 KiB** (`pdf_common.rs`). The truncation notice explicitly tells the model to narrow the result using page-range selection instead of reading whole large documents.
- `pdf_read`'s reported line count reflects the text *after* truncation.
- `pdf_read`'s loop also short-circuits once the accumulated page text passes the 100 KiB budget.
- Write errors embed a truncated JSON snippet of the offending payload in the error message.

---

6. Permissions and checkpointing

- **Permission categories.** `pdf_read` requires `file:read`; `pdf_write` requires `file:write`. These flow through the standard configurable allow/deny/ask rule engine and the file-path guards – the same surface as the plain `read`/`Write` tools. There are no PDF-specific permission rules.
- **Goal-loop write detection.** `LOOP_WRITE_TOOLS` (in `crates/ragent-agent/src/session/loop_state.rs`) includes `pdf_write`. Inside a `/loop` run, any call to this tool counts as a destructive write: it arms the pre-loop snapshot/checkpoint machinery (forced checkpoints gated on `spec.checkpoints` and `checkpoint_timeout_secs`) and participates in the loop change summary/diffstat.
- **Auto-approval interplay.** `--yes` (alias `--no-prompt`) and YOLO mode auto-approve ordinary permission prompts, but forced loop checkpoints still fire for this write tool while loop checkpoints are enabled.
- **Visibility vs executability.** Hiding a tool (by name, via a config overlay) removes it from the model's tool definitions, but a call by name still executes.

---

7. Implementation notes

- All parsing and writing is pure Rust – no LibreOffice/soffice subprocess, no Word installation, no headless converters.
- Underlying crates (from `crates/ragent-tools-extended/Cargo.toml`): `printpdf` (PDF writing), `pdf-extract` (PDF reading, vendored at `vendor/pdf-extract`), and `lopdf` (PDF parsing and the per-page fast path).
- `pdf-extract` is a workspace-patched dependency: the root `Cargo.toml` declares `pdf-extract = "0.10"` and patches it to `vendor/pdf-extract`. Extraction is whole-document-in-memory (`extract_text_from_mem`); encrypted variants exist upstream but are unused.
- Both `pdf_read` and `pdf_write` execute on the tokio blocking pool (`spawn_blocking`), so large document work does not stall the async runtime.
- PDF writes do not take their own file snapshots inside the tool; the snapshot/undo machinery that covers them is the session-level snapshot path and, inside goal loops, the forced-checkpoint machinery described in Section 6.

---

8. Worked examples

The examples below show the JSON arguments an agent passes to each tool. In the TUI you would normally just ask for the outcome ("read the table from report.pdf"); the model issues these calls itself.

8.1 Read a PDF as text

```
{ "path": "docs/report.pdf" }
```

8.2 Read a page range

```
{ "path": "papers/attention.pdf", "start_page": 3, "end_page": 7 }
```

8.3 Inspect PDF metadata

```
{ "path": "docs/spec.pdf", "format": "metadata" }
```

8.4 Read structured per-page JSON

```
{ "path": "reports/audit.pdf", "format": "json" }
```

8.5 Create a PDF report

```
{
  "path": "reports/audit.pdf",
  "content": {
    "title": "Dependency audit",
    "elements": [
      { "type": "heading", "level": 1, "text": "Summary" },
      { "type": "paragraph", "text": "All direct dependencies pass cargo-deny." },
      { "type": "table",
        "headers": ["Crate", "Licence", "Status"],
        "rows": [["pdf-extract", "MIT", "ok"], ["printpdf", "MIT", "ok"]] }
    ]
  }
}
```

8.6 Read a PDF and drive a document task

```
ragent run --agent general "read assets/pdf/Profile.pdf and summarise the candidate's experience"
```

Sample PDF files for manual testing live in `assets/pdf/` (`Profile.pdf`, `vibeSDLC.pdf`).

---

9. Commands and configuration reference

TUI

Command	Effect
`/tools`	Show the `/tools` help
`/tools list`	List every registered tool with its source and description

Configuration (`ragent.json`)

There are no PDF-specific keys; all behaviour (formats, payloads, limits) is fixed by the tool implementations. The tools are registered and visible by default.

Permissions

Category	Tools
`file:read`	`pdf_read`
`file:write`	`pdf_write`

Configure allow/deny/ask rules for these categories in the `permissions` array of `ragent.json`, exactly as for the plain file tools.

---

10. Testing and sample files

- Behavioural coverage for the PDF tools is indirect: the `/tools` listing includes the PDF tool names, the TUI slash-command test exercises `/tools`, and the step-log input summary for `pdf_read` is locked in by `crates/ragent-tui/tests/test_tool_display.rs`.
- Sample asset files (`assets/pdf/Profile.pdf`, `assets/pdf/vibeSDLC.pdf`) are informal manual fixtures; nothing in `tests/` references them, so they are not CI coverage.
- `vendor/pdf-extract` is a workspace-patched dependency used by `pdf_read` (and MasterFetch's PDF path); it extracts text fully in memory with no external binary.

---

11. Related documentation

- `docs/howtos/reactagent.md` – the core per-turn agent loop that executes these tools.
- `docs/howtos/loopprogramming.md` – goal loops; relevant for how `pdf_write` participates in forced checkpoints.
- `TUI-QUICKSTART.md` – quick reference for slash commands including `/tools`.
- `crates/ragent-tools-extended/DOCS.md` – curated descriptions of the extended tool families.
- `SPEC.md` – full configuration schema.